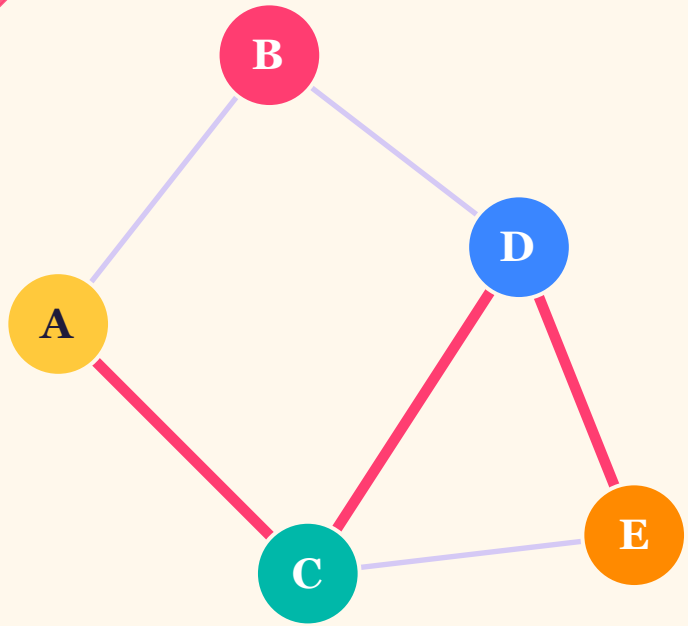
Design & Analysis of Algorithms

Dijkstra's Shortest Path Algorithm

How to find the quickest route in a weighted graph



Anushka Sarode
PRN: 25070521076 | Section: B

Why do we need it?

You are at your hostel and want to reach college. There are many routes. Which one is the shortest?

That question is called the shortest path problem, and Dijkstra's algorithm solves it.

01

Many routes

Even a small map has lots of ways to go from one place to another.

02

Different lengths

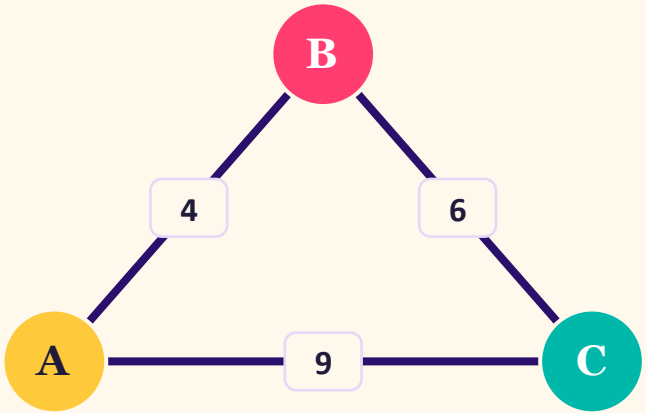
Every road has its own distance, time or cost.

03

One best answer

We want the route whose total is the smallest.

A quick look at graphs



Cost of A to C directly = 9
Cost of A to B to C = 4 + 6 = 10

Node (vertex)

A place, like a city or a junction.

Edge

A road that joins two nodes.

Weight

The length (or cost) of that road.

Path cost

The sum of all weights along a path.

The main idea

Always go to the closest place you have not finished yet.

This is called a greedy strategy.

We keep two lists

Visited nodes (distance is final) and unvisited nodes (distance may still improve).

The closest one is safe

No other road can reach it cheaper, because roads never have negative length.

Then we look around

From the new node, check if its neighbors can be reached in a shorter way.

The algorithm in 4 steps

1

Set up

Start = 0, all others = infinity (we don't know yet).

2

Pick closest

Take the unvisited node with the smallest distance.

3

Relax

Check each neighbor. Found a shorter way? Update it.

4

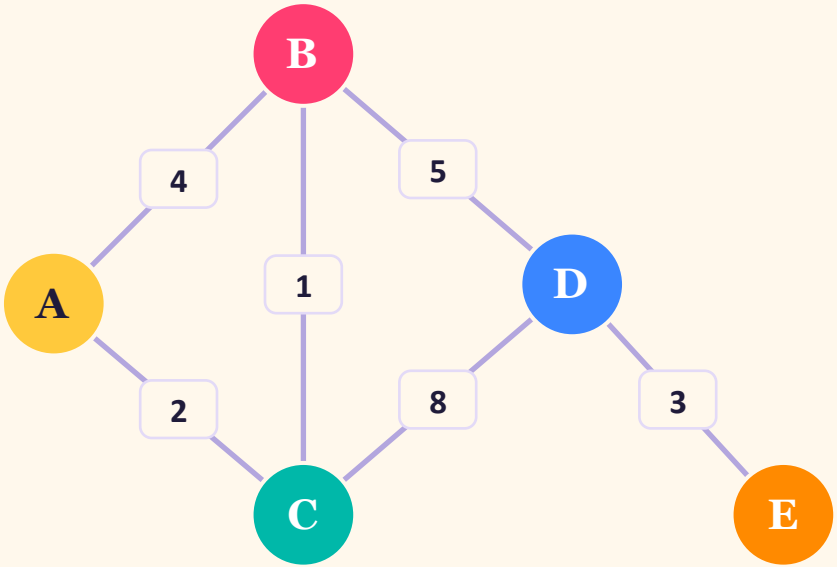
Repeat

Mark the node visited. Stop when all are visited.

"Relaxing" just means: is the path through this node shorter than what I know so far?

Worked example: step by step

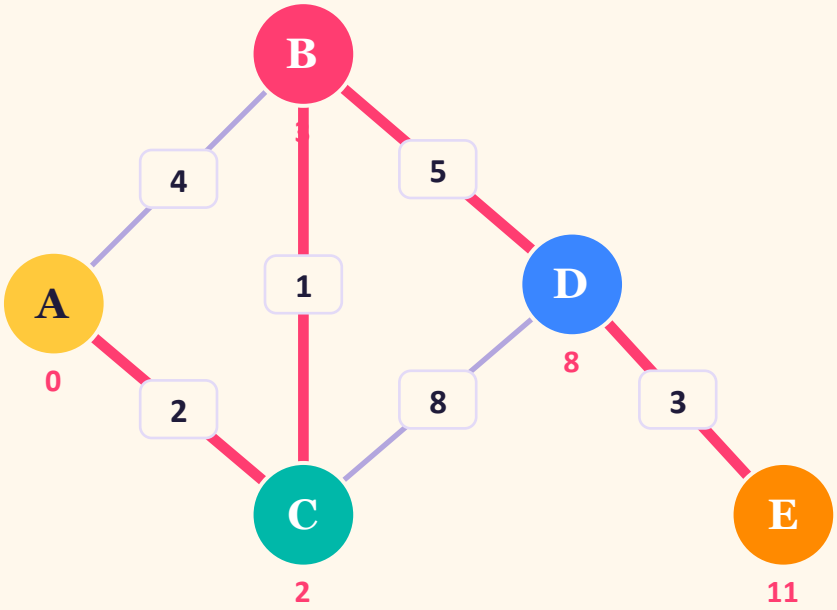
Start node: A



Picked	A	B	C	D	E
start	0	∞	∞	∞	∞
A	0	4	2	∞	∞
C	0	3	2	10	∞
B	0	3	2	8	∞
D	0	3	2	8	11

Orange = value updated in that round. Each round we pick the smallest unvisited value.

Worked example: the result



2 — **A to C**
A → C

3 — **A to B**
A → C → B

8 — **A to D**
A → C → B → D

11 — **A to E**
A → C → B → D → E

Notice: A to B is 3 through C, which beats the direct road of 4.

The pseudocode

```
dist[start] = 0
dist[others] = ∞
queue = all nodes

while queue is not empty:
  u = node with smallest dist
  remove u from queue
  for each neighbor v of u:
    if dist[u]+w(u,v) < dist[v]:
      dist[v] = dist[u]+w(u,v)
```

Lines 1 to 3

Set up distances and the queue.

Line 5 to 7

Pick the closest node and take it out.

Lines 8 to 10

Relax: update neighbors if a shorter path exists.

How fast is it?

	Simple array	Min-heap
Find closest node	O(V) each time	O(log V)
Update a distance	O(1)	O(log V)
Total time	**$O(V^2)$**	**O((V+E) log V)**
Extra space	O(V)	O(V)
Works best for	Dense graphs	Large, sparse graphs

V = number of nodes, E = number of edges

Limits: negative weights need Bellman-Ford. All pairs need Floyd-Warshall.

Wrapping up

Where it is used

- GPS and maps
- Internet routers (OSPF)
- Flight and travel apps
- Path finding in games

Remember

- It is greedy: closest node first
- Relax edges to find shortcuts
- Needs non-negative weights
- Time: O((V+E) log V) with a heap

Thank you!

Questions are welcome